

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Team Name: Technologia

Team Member:

1.	Lam Yean Chin
2.	Hee Zhi Ding
3.	Tang Jac Tze
4.	Ng Mun Shen
5.	Fu Ly Ning

Table Of Content

SYSTEM ANALYSIS DOCUMENTATION (SAD)	1
Introduction	2
Purpose	2
Background	3
Previous State / Problem	3
What ClearPath Introduces	3
Target Stakeholders	4
System Architecture & Design	4
High-Level Architecture	4
LLM as Service Layer — Dependency Diagram	5
How Prompts Are Constructed	5
What Goes Into the Context Window	6
How Responses Are Received and Parsed	6
Token Limit Enforcement	6
Major API Calls Between Components	6
Component Interaction Map	7
Sequence Diagram — Full Triage Flow (Happy Path)	8
Technology Stack	8
Key Data Flows	10
State Schema (Logical Data Model)	10
Functional Requirements & Scope	12
Minimum Viable Product	12
Non-Functional Requirements (NFRs)	12
Out of Scope / Future Enhancements	13
Monitor, Evaluation, Assumptions & Dependencies	14
Technical Evaluation	14
Priority Matrix	14
Monitoring Plan	15
Assumptions	15
External Dependencies	15
Project Management & Team Contributions	16
Project Timeline (10-Day Sprint)	16
Team Member Roles	17
Recommendations	17

Introduction

This document introduces ClearPath, a strategic AI-powered solution to the fragmented, manual emergency triage processes in Malaysian public hospital emergency departments. ClearPath is a response to Domain 1 of UMHackathon 2026: AI Systems & Agentic Workflow Automation.

Purpose

This System Analysis Document highlights the technical scope and design decisions behind ClearPath's development. The key elements covered are:

- **Architecture:** A single-page React application powered by Z.AI GLM as a central reasoning service, with Vite as the development and proxy server layer
 - **Data Flows:** How unstructured multilingual patient complaints travel from the intake form, through the GLM reasoning API, and back as structured JSON that drives downstream workflow orchestration
 - **Model Process:** End-to-end workflow from patient intake → GLM triage → workflow task orchestration → live dashboard update
 - **Role of Reference:** This document serves as the primary reference for developers, QA testers, and judges to understand ClearPath's architecture, component interactions, and design decisions
-

Background

Malaysian public hospitals, particularly government hospitals under the Ministry of Health (KKM), handle over 6 million emergency department visits per year. Triage is a critical first step: the wrong classification can cost lives. Currently, triage nurses manually assess patients, consult paper-based triage guidelines, and verbally coordinate with multiple departments (bed management, labs, pharmacy, doctors).

This process is:

- Slow, average triage wait: 20–45 minutes in busy EDs
- Error-prone, language barriers, handwriting errors, miscommunication
- Linguistically fragmented, patients in Malaysia speak BM, English, Tamil, Mandarin, or Manglish code-switches

ClearPath replaces this manual coordination with a GLM-powered triage engine that reads unstructured multilingual input and generates structured clinical decisions and workflow orders in under 5 seconds.

Previous State / Problem

No existing digital triage tool handles Manglish or Bahasa Malaysia natively. Hospital systems like HIS (Hospital Information System) are administrative tools, not clinical reasoning engines. Nurses rely on memory, printed triage charts, and telephone calls.

What ClearPath Introduces

- First-pass AI triage using Malaysian Emergency Triage categories (P1–P5)
- Multilingual NLP (BM, English, Manglish) without translation preprocessing
- Automated downstream workflow orchestration: 6 parallel task streams triggered by one GLM call

Target Stakeholders

Stakeholder	Role	Expectations
Triage Nurse	Enters patient complaint and vitals; reviews GLM triage recommendation	Fast result (<5s), clear triage level (P1–P5), actionable workflow orders
Emergency Doctor	Receives alert from the system; reviews clinical reasoning	Accurate specialty routing, red flag highlights, estimated urgency
Hospital Administrator	Monitors dashboard: bed counts, queue, activity feed	Real-time visibility into ED load, occupancy rates, patient flow
Development Team	Build and maintain the system	Clear GLM integration contract, modular components, documented architecture
QA Team	Validate GLM output accuracy, error handling, UI correctness	Defined API schema, reproducible demo scenarios, edge case documentation

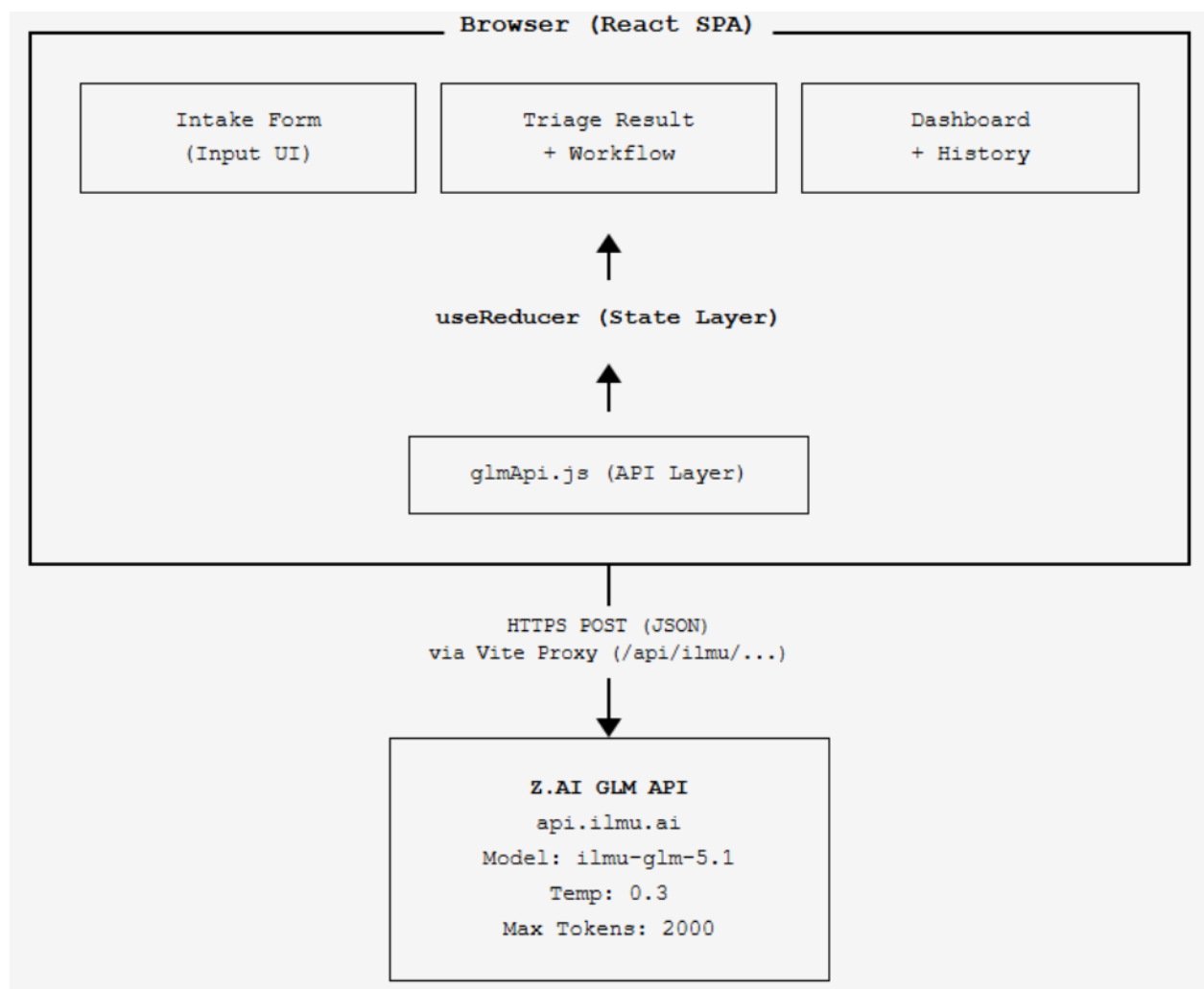
System Architecture & Design

High-Level Architecture

Type	Details
------	---------

System	Web Application (Browser-based, Desktop-first)
Architecture	Single-Page Application (SPA) with GLM-as-a-Service integration via Vite reverse proxy

ClearPath is structured as a client-side React SPA. There is no traditional backend, all business logic, state management, and workflow orchestration runs in the browser. The only external service call is to Z.AI's GLM API (<https://api.ilmu.ai>), accessed via a Vite development proxy to avoid CORS restrictions.



LLM as Service Layer — Dependency Diagram

How Prompts Are Constructed

In `glmApi.js`, the `callGLMTriage()` function builds a two-part message:

1. System Prompt (fixed, ~350 tokens): Instructs GLM to act as a Malaysian ED triage AI. Defines the P1–P5 classification framework, specifies the exact JSON output schema, and instructs GLM to handle multilingual input and flag ambiguities.
2. User Message (dynamic, ~50–450 tokens): Constructed from patient intake fields:

PATIENT INTAKE:

Chief Complaint: [free text — BM/EN/Manglish]

Age: [number]

Known Allergies: [text]

Current Medications: [text]

Vitals:

Blood Pressure: [BP]

Heart Rate: [HR] bpm

SpO2: [SpO2]%

Temperature: [temp]°C

What Goes Into the Context Window

Component	Approx. Tokens	Content
System Prompt	~350 tokens	Triage framework, JSON schema, multilingual instruction
User Message	~50–450 tokens	Patient complaint + optional vitals
Total	~400–800 tokens	Well within <code>ilmu-glm-5.1</code> 's context limit

How Responses Are Received and Parsed

1. GLM responds with `choices[0].message.content` (a JSON string)
2. `parseGLMResponse()` strips any accidental markdown code fences (````json`)
3. `JSON.parse()` converts to a JavaScript object
4. Validation checks that `triage_level` and `workflow_orders` exist
5. Valid result is dispatched to the `useReducer` state via `SET_RESULT` action

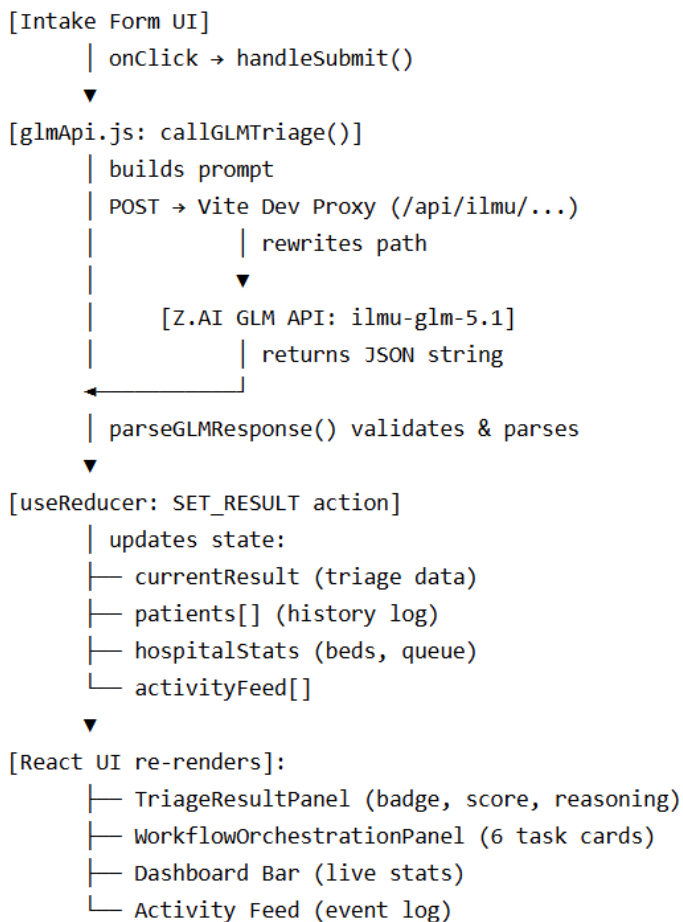
Token Limit Enforcement

- Input is not chunked (patient intake is naturally short)
- max_tokens: 2000 caps the GLM response size
- A 30-second AbortController timeout protects against hung requests
- User inputs are not truncated — the UI design (textarea) naturally limits input length

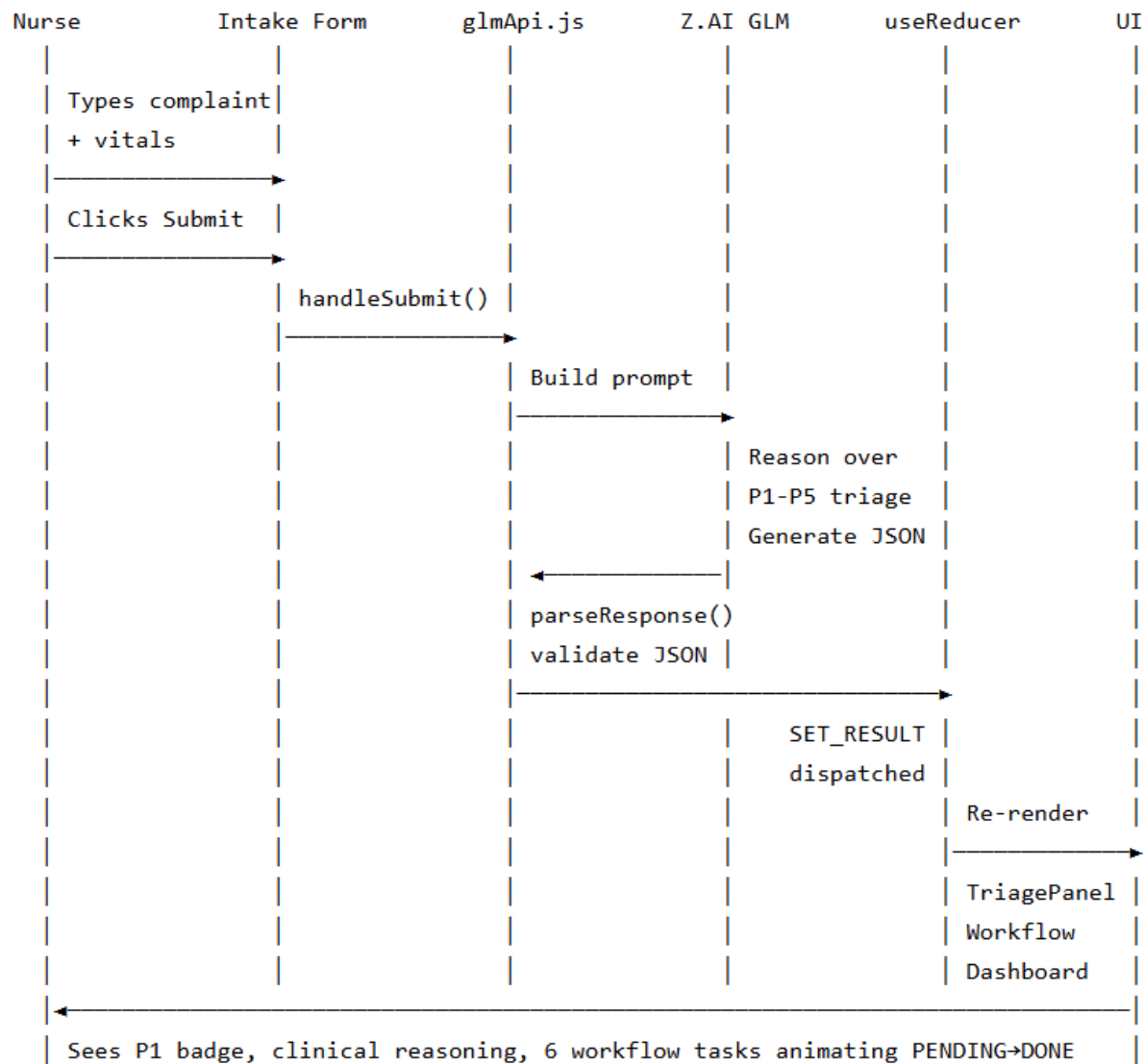
Major API Calls Between Components

Caller	Endpoint	Method	Purpose
glmApi.js (via Vite proxy)	/api/ilmu/v1/chat/completions → https://api.ilmu.ai/v1/chat/completions	POST	Send patient intake, receive structured triage JSON

Component Interaction Map



Sequence Diagram — Full Triage Flow (Happy Path)

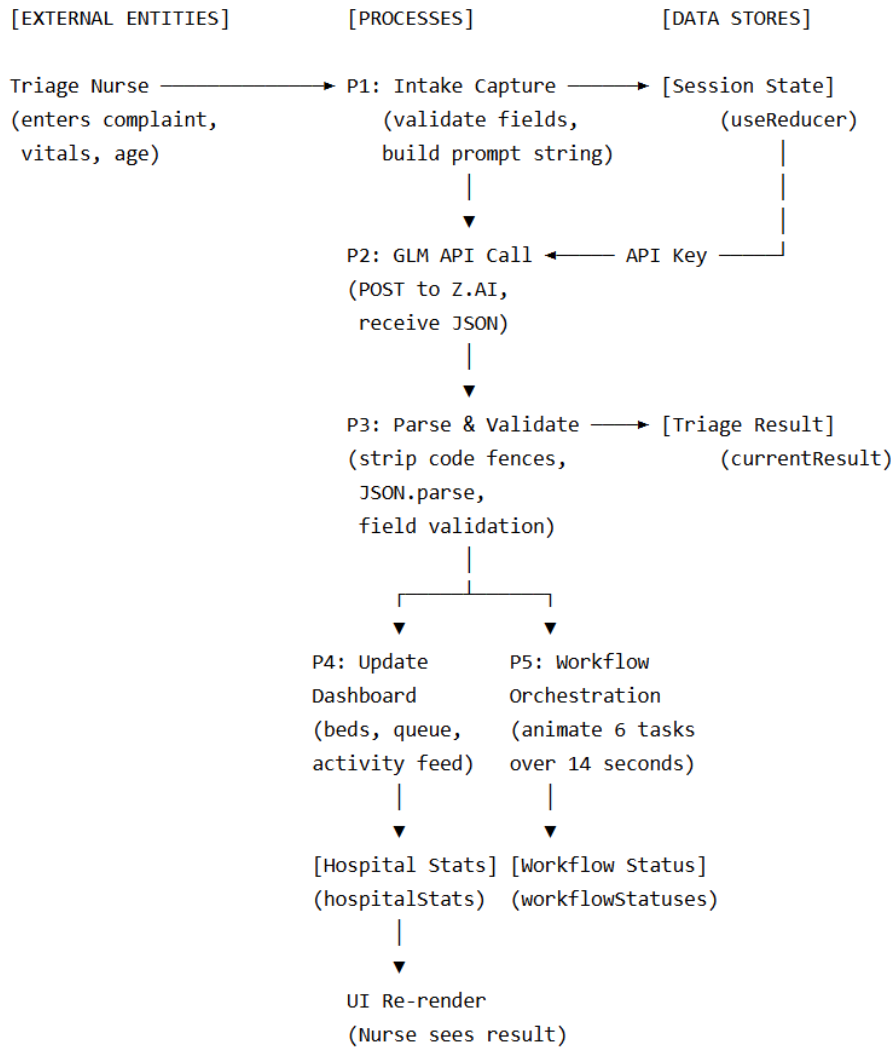


Technology Stack

Layer	Technology	Justification
Frontend	React 19 + JSX	Component-based UI, useReducer for predictable state, ecosystem maturity

Styling	Tailwind CSS v4	Utility-first for rapid dark-mode medical UI; design tokens via CSS variables
Build Tool	Vite 5	Fast HMR dev server; built-in reverse proxy for GLM API CORS handling
AI Reasoning	Z.AI GLM (ilmu-glm-5.1)	OpenAI-compatible API; strong multilingual capability (BM/EN/Manglish)
State Management	React useReducer	No backend needed; deterministic state transitions; easy to audit
Audio	Web Audio API	Native browser audio for P1/P2 alert beeps — no library dependency
Fonts	JetBrains Mono + DM Sans	Monospace for clinical data readability; sans-serif for UI
Deployment	Vite Dev Server (localhost) / Vercel-compatible static build	npm run build produces deployable static assets

Key Data Flows



Data Integrity Notes:

- No Black Hole: every user action triggers a state update that feeds back to the UI
- No data leaves the browser except the single GLM API POST request
- API key is stored only in React state (never in localStorage, never in source code)

State Schema (Logical Data Model)

```

AppState {
  apiKey: string           // Runtime only, never persisted
  intake: {
    complaint: string      // Free text — BM/EN/Manglish
    age: string
    allergies: string
    medications: string
    vitals: { bp, hr, spo2, temp } // All optional
  }
}
  
```

```

}
currentResult: TriageResult | null
patients: Patient[]           // Session-scoped history log
hospitalStats: {
  totalPatientsToday: number
  beds: { resus, critical, general } // { total, available }
  queue: { P1, P2, P3, P4, P5 }
  avgWait: { P1, P2, P3 }
}
workflowStatuses: {           // Per-patient, resets on new triage
  bed_assignment: 'PENDING'|'IN PROGRESS'|'DONE'
  doctor_alert:  'PENDING'|'IN PROGRESS'|'DONE'
  lab_orders:    'PENDING'|'IN PROGRESS'|'DONE'
  imaging_orders: 'PENDING'|'IN PROGRESS'|'DONE'
  pharmacy_prep: 'PENDING'|'IN PROGRESS'|'DONE'
  nursing_actions: 'PENDING'|'IN PROGRESS'|'DONE'
}
activityFeed: ActivityItem[]   // Max 20 items, LIFO
isDemoRunning: boolean
demoStep: number              // -1 = not running
detailPatient: Patient | null
activeTab: 'dashboard' | 'history'
}
TriageResult {
  triage_level: 'P1'|'P2'|'P3'|'P4'|'P5'
  triage_score: number          // 1–100
  chief_complaint: string
  red_flags: string[]
  clinical_reasoning: string
  confidence: 'high'|'medium'|'low'
  ambiguity_flags: string[]
  workflow_orders: {
    bed_assignment: { ward, bed_type, urgency }
    doctor_alert: { specialty, message, urgency }
    lab_orders: { test, urgency }[]
    imaging_orders: { type, body_part, urgency }[]
    pharmacy_prep: { medication, reason }[]
    nursing_actions: string[]
  }
}
estimated_wait_minutes: number
}

```

Functional Requirements & Scope

Minimum Viable Product

#	Feature	Description
1	Patient Intake	Nurse enters free-text complaint (BM/EN/Manglish), optional age, allergies, medications, and vitals (BP, HR, SpO2, Temperature). Supports partial data entry — vitals are optional.
2	GLM Triage Classification	System sends structured prompt to Z.AI GLM and receives a P1–P5 triage level, triage score (1–100), clinical reasoning, red flags, and confidence rating.
3	Workflow Orchestration	Based on GLM's workflow_orders, 6 tasks animate through PENDING → IN PROGRESS → DONE: Nursing Actions, Bed Assignment, Doctor Alert, Lab Orders, Imaging, Pharmacy Prep.
4	Live Hospital Dashboard	Real-time display of bed availability (Resus/Critical/General), queue by triage level (P1–P5), average wait times, and live activity feed.
5	Demo Mode	One-click auto-run of 5 pre-built clinical scenarios (P1 cardiac, P2 head trauma, P2 tropical fever, P4 sprain, P5 mild pain) to showcase full system capability.

Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Performance	GLM API must respond within 5000ms under normal load	30s AbortController timeout; loading spinner shown during API call; avg observed response ~2.6s
Reliability	API failure must not crash the app; user must receive a clear error	try/catch wraps all API calls; error state dispatched and displayed; dismiss button allows retry

Token Latency	GLM context must not exceed model limits	System prompt ~350 tokens + user message ~450 tokens = ~800 tokens total; max_tokens: 2000 for response
Security	API key must never appear in source code or network logs as plaintext in URL	API key entered at runtime in a password-type input; sent as Authorization: Bearer header only
Maintainability	Logic separated from UI	glmApi.js (API), demoData.js (data), App.jsx (UI + state); reducer isolated in switch statement
Scalability	System must handle repeated rapid submissions	Stateless API calls; each submission is independent; demo mode uses async/await with sequential delays
Cost Efficiency	Token usage per session should be minimal	Short, fixed-structure system prompt; user inputs are naturally brief; no repeated context injected
Accessibility	System must remain usable if audio is unavailable	Web Audio API wrapped in try/catch; alert beep failure is silent — UI visual cues always present

Out of Scope / Future Enhancements

- User Authentication: Login, role-based access (nurse vs. doctor vs. admin views)
- Backend Persistence: Database storage of patient records beyond browser session
- Real HIS Integration: Actual API calls to lab systems, bed management software, pharmacy dispensing systems
- Mobile Responsiveness: Optimised layout for tablet or phone use by bedside nurses
- EHR Integration: Connecting triage results to Electronic Health Records (MyMedic/MySejahtera)
- Multi-department Collaboration: Real-time multi-user dashboard where nurse and doctor see updates simultaneously
- Analytics & Reporting: Triage pattern analytics, average response time reports for hospital management

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation

Grayscale Rollout & A/B Testing: For production deployment, ClearPath would follow a phased rollout:

- Phase 1 (5%): Deploy to 1 triage terminal in a single ED. Monitor triage accuracy vs. nurse manual classification. Collect ambiguity flag rate.
- Phase 2 (25%): Expand to 5 terminals if Phase 1 shows $\geq 85\%$ triage alignment with nurse judgment.
- Phase 3 (100%): Full ED deployment. A/B testing between GLM temperature settings (0.2 vs. 0.4) to compare clinical reasoning quality.

Emergency Rollback (ER) & Golden Release:

- If GLM API returns $>10\%$ error rate in a 5-minute window, system auto-displays degraded mode: intake form remains functional but triage result panel shows "AI Unavailable — Manual Triage Required"
- The last known stable build is tagged as the Golden Release in GitHub; rollback is triggered by reverting to that tag and redeploying the static build

Priority Matrix

Priority	Trigger Condition	Action
P0 — Critical	GLM API completely unreachable for >2 minutes	Display "AI Offline" banner across all terminals; notify IT immediately; switch to manual triage protocol
P1 — High	GLM JSON parse failure rate $>5\%$ in 10 minutes	Alert development team; check if GLM model version changed; revert system prompt if needed
P2 — Medium	Average GLM response time $>5000\text{ms}$ sustained for 5 minutes	Display extended loading warning to nurse; log response times for post-incident review
P3 — Low	Bed count display inconsistency	Flag for next sprint review; no immediate action required

Monitoring Plan

Metric	Monitoring Method	Alert Threshold
GLM API response time	Browser Performance API + console logging	Alert if >5000ms
GLM JSON parse success rate	Error boundary in <code>parseGLMResponse()</code> + <code>console.error</code>	Alert if >3 consecutive failures
Bed count validity	Guard clause check: available never < 0	Console.warn if decrement attempted on 0
Demo mode completion	try/catch per scenario; error count logged	Alert if >2/5 scenarios fail

Assumptions

1. Triage nurses have access to a modern desktop browser (Chrome/Edge) on a hospital workstation
2. Hospital workstations have stable internet connectivity to reach `api.ilmu.ai`
3. The Z.AI GLM API (`ilmu-glm-5.1`) remains available and maintains its OpenAI-compatible response format
4. Patient descriptions entered by nurses are at least minimally descriptive (not completely empty)
5. ClearPath is used as a decision support tool — final triage decisions remain the nurse's clinical responsibility

External Dependencies

Tool	Purpose	Risk	Mitigation
Z.AI GLM API (<code>api.ilmu.ai</code>)	Core reasoning engine — multilingual NLP, P1–P5 classification, workflow order generation	High — if API is down or rate-limited, core product fails	30s timeout with graceful error UI; degraded mode shows "AI Unavailable"; retry is one click

Vite Dev Server	Reverse proxy to avoid CORS on GLM API in development	Low — development only; production build uses direct API calls or separate proxy	Static build removes Vite dependency; can deploy behind Nginx proxy in production
Web Audio API	P1/P2 alert beep	Low — not supported on all browsers	Wrapped in try/catch; silent failure; visual alerts always present
Google Fonts (CDN)	JetBrains Mono + DM Sans typography	Low — cosmetic only	Fallback fonts defined in CSS (monospace, sans-serif)

Project Management & Team Contributions

Project Timeline (10-Day Sprint)

Days	Phase	Activities
Day 1–2	Planning & Design	Problem framing, GLM prompt engineering strategy, UI wireframes, JSON schema design, ARCHITECTURE.md drafted
Day 3–4	Core Build	glmApi.js integration with Z.AI, useReducer state architecture, patient intake form, basic triage result display
Day 5–6	Feature Completion	Workflow orchestration panel, hospital dashboard (beds/queue/activity feed), patient history log, demo scenarios in demoData.js
Day 7	Polish & UX	P1 screen glow, audio alert beep, typewriter clinical reasoning effect, animation system, responsive layout

Day 8	Testing & QA	Manual test of all 5 demo scenarios, edge case testing (no vitals, timeout, malformed JSON), QATD documentation
Day 9	Documentation	ARCHITECTURE.md finalised, README.md written, SAD and QATD completed
Day 10	Demo Prep	Demo Mode validated end-to-end, slide deck prepared, verbal walkthrough rehearsed

Team Member Roles

Member	Role	Responsibilities
(Team Lead / Full-Stack)	System Architect & GLM Integration	GLM prompt engineering, glmApi.js, vite.config.js proxy, overall architecture
(Frontend Lead)	React UI & State Management	App.jsx component structure, useReducer reducer, all UI panels and animations
(Product & Data)	Demo Data & Documentation	demoData.js clinical scenarios, ARCHITECTURE.md, README.md, SAD, QATD
(QA & Design)	Design System & Testing	index.css design tokens, animation system, manual test execution, QATD test cases

Recommendations

Area	Recommendation	Rationale
Code Modularity	Split App.jsx (1,342 lines) into a components/ folder	Improves maintainability and scores higher on Code Modularity judging criterion

Testing	Add Vitest unit tests for <code>parseGLMResponse()</code> and the reducer	Addresses the zero-test gap in QA judging criteria (10% of total score)
Caching	Cache identical complaint+vitals combos for 5 minutes	Avoids duplicate GLM calls if nurse accidentally double-submits
Backend (Post-MVP)	Add a lightweight Express/FastAPI backend for session persistence	Enables multi-device dashboard sharing and permanent patient records
Prompt Versioning	Store system prompt version in a constant	Makes it easy to A/B test prompt variations and roll back if GLM behaviour changes
Real API Integration	Implement webhook stubs for bed system and lab system	Demonstrates genuine workflow integration capability for final round judges

ClearPath — Built for UMHackathon 2026 | Domain 1: AI Systems & Agentic Workflow Automation